| Task – 1002 | 03-08-2026 |
|---|---|

# Intelligent Email Processing & Automated Customer Response System With MCP.

## 1. Problem Statement

A financial institution receives customer emails related to:

- Account balance enquiry
- Credit card usage enquiry
- Account statement requests
- Multiple requests within a single email

Currently, customer support agents manually:

1. Read and understand customer emails
2. Identify customer intent and sentiment
3. Validate account/credit card details
4. Uses core banking applications
5. Draft response emails

This manual process is time-consuming, error-prone and difficult to scale.

The objective is to design and develop an AI-powered intelligent automation system using:

- Python
- APIs
- LLM (Large Language Model) or SLM
- Llama
- Agentic AI architecture

The system should automatically process incoming emails, identify customer requirements, fetch required information from core systems and generate appropriate response emails.

---

## 2. Objective

The candidate is expected to:

- Analyze customer emails using NLP/LLM
- Detect sentiment and customer intent from mail body.
- Download the attached document (pdf, image(any format)).
- Extract account/card-related information from downloaded attachments.
- Handle multiple requests in a single email
- Validate extracted data
- Decide which backend API(s) to call
- Fetch data from mocked/simulated core APIs
- Generate professional customer responses
- Implement agentic workflow orchestration
- Prepare architecture and technical documentation

---

## 3. Business Scenarios

### Scenario 1 – Account Balance Enquiry

**Customer Email**

Please provide my savings account balance for account number 1234567890.

**Expected System Behaviour**

- Identify intent: Account Balance Enquiry
- Extract account number
- Call account balance API

- Generate response email with balance details

---

**Scenario 2 – Credit Card Usage Enquiry**

**Customer Email**

Please share my last 5 credit card transactions for card 4567XXXX8901.

**Expected System Behaviour**

- Identify intent: Credit Card Usage
- Extract card number
- Call card transactions API
- Generate response with transaction summary

---

**Scenario 3 – Statement Request**

**Customer Email**

Kindly send my bank statement for April 2026.

**Expected System Behaviour**

- Identify intent: Statement Request
- Extract statement period
- Call statement API
- Generate response with statement attachment/reference

---

**Scenario 4 – Multiple Requests in One Email**

**Customer Email**

Please share my account balance for account 1234567890 and also send last 3 transactions of my credit card 987654321.

**Expected System Behaviour**

- Detect multiple intents

- Execute multiple API calls
- Consolidate responses into single email

---

**Scenario 5 – Partial Validation Failure**

**Customer Email**

Please provide balance for account 1234567890 and card transactions for card 999999999.

**Backend Validation**

- Account number valid
- Credit card invalid

**Expected Response**

- Provide account balance successfully
- Inform customer that card details are invalid
- Request customer to recheck card information

---

## 4. Functional Requirements

### 4.1 Email Processing

System must:

- Read email content
- Clean and preprocess text
- Detect language (optional enhancement)

---

### 4.2 Intent Detection

System should identify:

- Balance enquiry
- Credit card usage enquiry

- Statement request
- Multiple intents

---

### 4.3 Sentiment Analysis

Identify:

- Neutral
- Positive
- Negative/Urgent

Example:

- Angry customer
- Escalation tone
- Urgent request

---

### 4.4 Entity Extraction

Extract:

- Account number
- Credit card number
- Date range
- Transaction count
- Customer name (optional)

---

### 4.5 API Decision Engine

System should determine:

- Which APIs need to be called
- Sequence of API execution

- Parallel processing opportunities

---

### 4.6 Core API Integration

Candidate should create mock APIs for:

| API | Purpose |
|---|---|
| GetAccountBalanceAPI | Fetch account balance |
| GetCardTransactionsAPI | Fetch card transactions |
| GetStatementAPI | Fetch statements |
| ValidateAccountAPI | Validate account number |
| ValidateCardAPI | Validate card number |

---

### 4.7 Response Generation

System should:

- Generate professional response email
- Handle partial success/failure
- Provide clarification requests where needed

---

### 4.8 Error Handling

System should handle:

- Invalid account/card number
- API failure
- Timeout
- Missing details
- Unsupported requests

---

## 5. Non-Functional Requirements

| Requirement | Description |
|---|---|
| Scalability | Support large volume of emails |
| Security | Mask sensitive data |
| Auditability | Log all API calls and decisions |
| Extensibility | Add new intents easily |
| Accuracy | High intent classification accuracy |
| Performance | Response within acceptable SLA |

---

## 6. Proposed Automation Architecture

### High-Level Workflow

Incoming Email

↓

Email Reader Module

↓

LLM/NLP Processing

↓

Intent Detection + Sentiment Analysis

↓

Entity Extraction

↓

Agent Orchestrator

↓

API Decision Engine

↓

Core Banking APIs

↓

Response Generator

↓

Final Email Response

---

## 7. Proposed Agentic AI Design

### Suggested Agents

| Agent | Responsibility |
|---|---|
| Email Understanding Agent | Read and summarize email |
| Intent Classification Agent | Identify customer requests |
| Sentiment Analysis Agent | Detect customer sentiment |
| Entity Extraction Agent | Extract account/card details |
| Validation Agent | Validate extracted data |
| API Selection Agent | Determine required APIs |
| Response Generation Agent | Draft final response |
| Audit/Logging Agent | Store execution logs |

---

## 8. Suggested Technology Stack

| Component | Suggested Technology |
|---|---|
| Programming Language | Python |
| LLM Framework | LangChain / LlamaIndex |

| Component | Suggested Technology |
|---|---|
| Model | LLaMA 3 |
| API Framework | FastAPI / Flask |
| Vector Store (Optional) | FAISS |
| Database | PostgreSQL / MongoDB |
| Agent Framework | CrewAI / LangGraph |
| Email Processing | IMAP / SMTP |
| Deployment | Docker |

---

### 9. Expected Deliverables from Candidate

Candidate must provide:

**Documentation**

- Solution architecture document
- Low-level design
- API specifications
- Agent workflow document
- Assumptions and limitations

---

**Development Deliverables**

- Python source code
- Mock APIs
- LLM integration
- Agent orchestration
- Sample test emails
- Logging framework
- Configuration files

- README document

---

**Demonstration**

Candidate should demonstrate:

- End-to-end workflow
- Multi-intent handling
- Partial validation failure handling
- Response generation
- API orchestration

---

## 10. Expected Folder Structure

project/

│

├── agents/

├── apis/

├── models/

├── prompts/

├── workflows/

├── utils/

├── tests/

├── config/

├── logs/

├── main.py

├── requirements.txt

└── README.md

---

## 11. Sample API Contract

**Validate Account API**

**Request**

```
{
  "account_number": "1234567890"
}
```

**Response**

```
{
  "status": "VALID",
  "customer_name": "John Doe"
}
```

---

## 12. Sample Expected Response Email

Dear Customer,

We have processed your request successfully.

Account Balance:

- Account Number: XXXX7890
- Available Balance: INR 1,25,000

Credit Card Request:
The credit card number provided could not be validated. Kindly recheck the card details and resend your request.

Thank you,
Customer Support Team

---

## 13. Optional Enhancements

Candidates may additionally implement:

- OCR for attachment reading
- Multi-language support

- Human-in-loop approval
- RAG-based banking policy lookup
- Voice/email omnichannel support
- Confidence scoring
- Real-time monitoring dashboard

---

## 14. Assumptions

- Core banking APIs are mocked/simulated
- Authentication can be simplified
- Email server integration can be simulated
- No real banking data is required

---

## 15. Interview Task Summary

The candidate is required to design and implement an AI-powered intelligent banking email automation system capable of:

- Understanding customer emails
- Detecting multiple intents
- Performing sentiment analysis
- Extracting banking entities
- Calling appropriate backend APIs
- Handling partial failures
- Generating professional response emails
- Using Python, APIs, LLMs, LLaMA and Agentic AI concepts

The solution should demonstrate scalability, modularity and production-ready design principles.